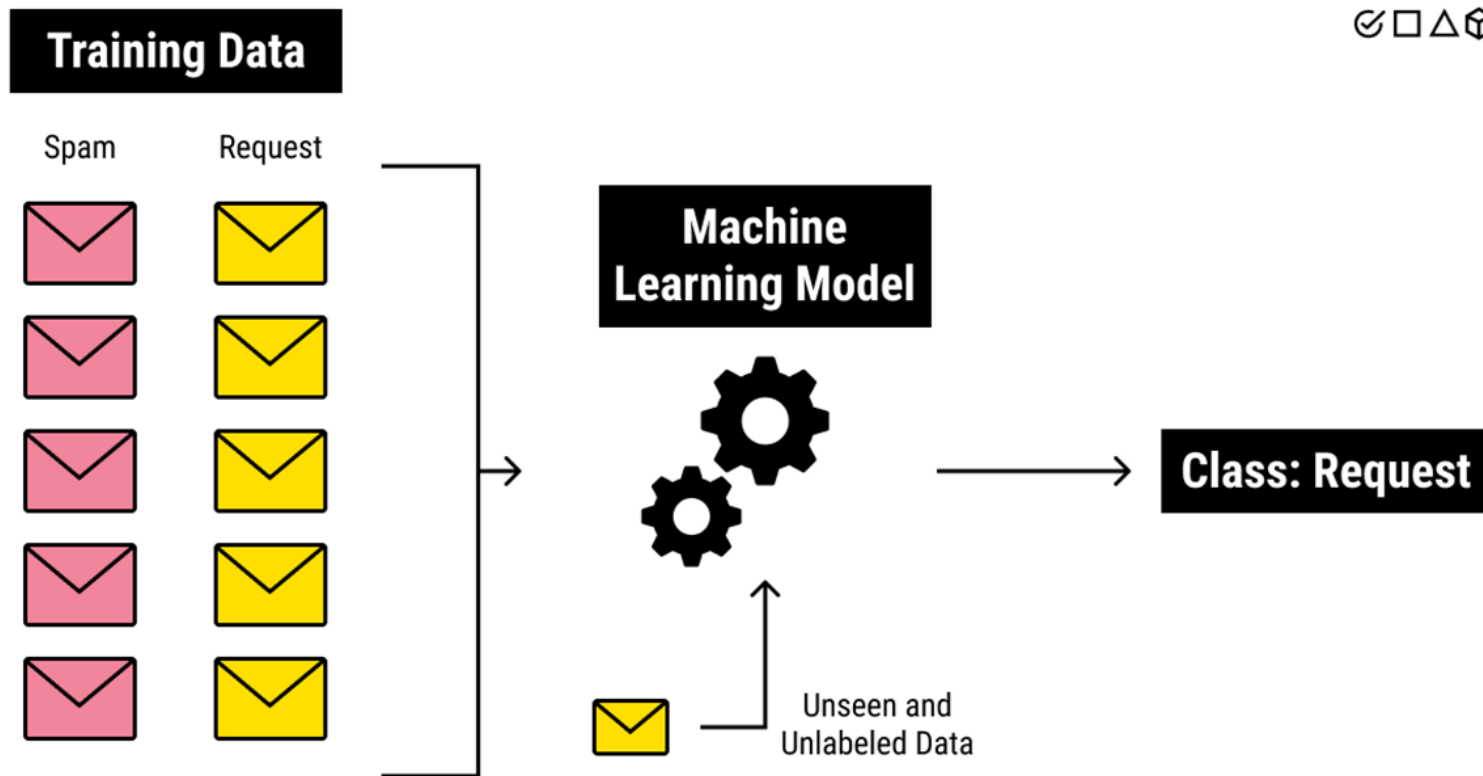


Classifier performance

Machine learning

Doc. Ing. Radim Kolar, Ph.D.
Department of Biomedical Engineering,
FEEC, Brno University of Technology

Spam – Nospam classifier



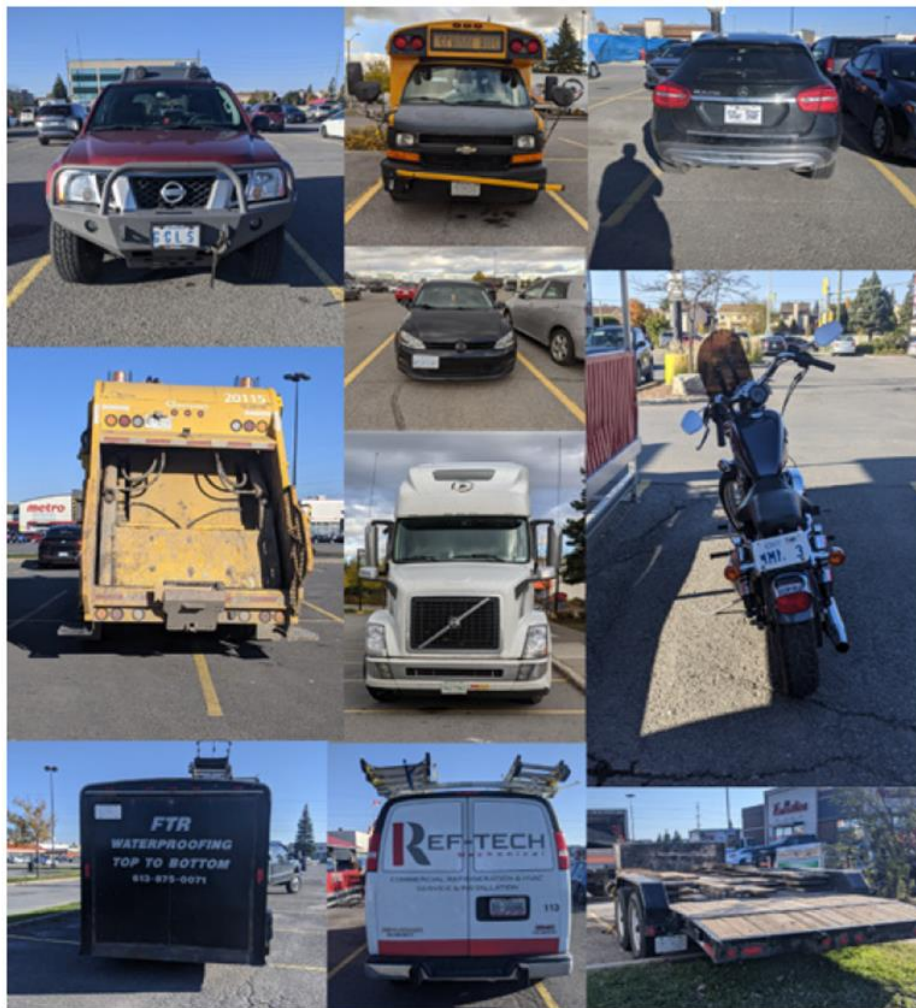
Importance of the data – car detection example

- For training we need a variety of images of cars from different angles and in real traffic scenario



Car detection example

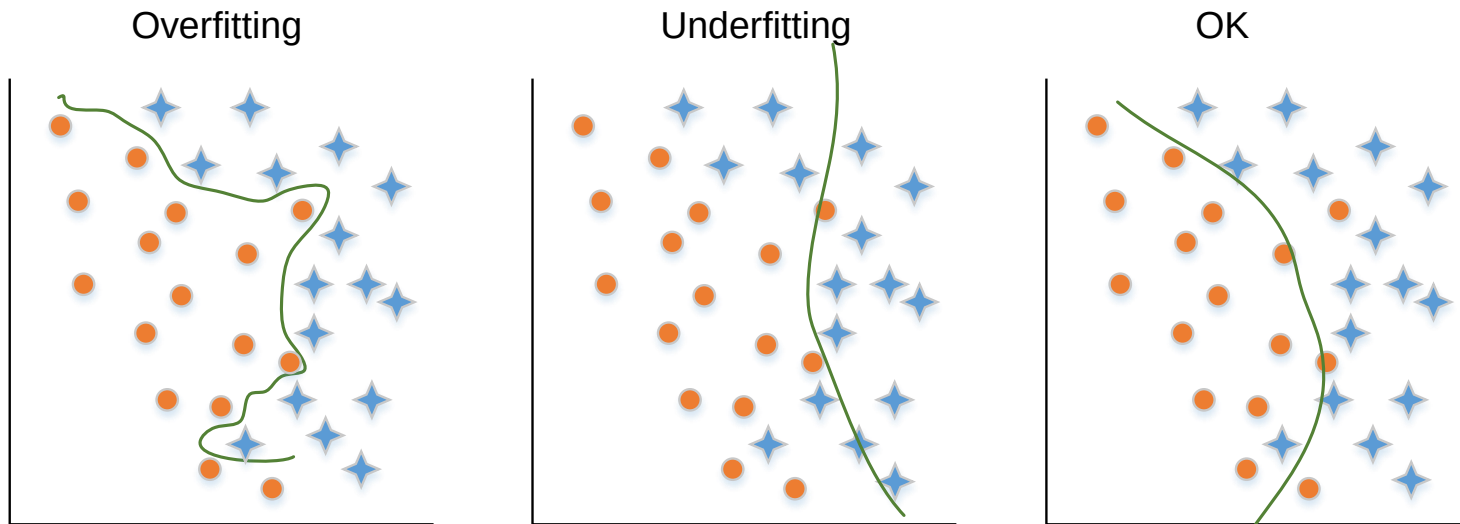
- However, during detection we can miss some cars



Overfitting, underfitting

Overfitting occurs when the classifier is “tightly fitted” to the training data points, and therefore loses its generalization ability for the test data. Because of this, the classification performance is lower in the test set when compared to the training set.

Underfitting - the classifier is not learned enough to respect the borders between classes.



Why we need to test classifier performance?

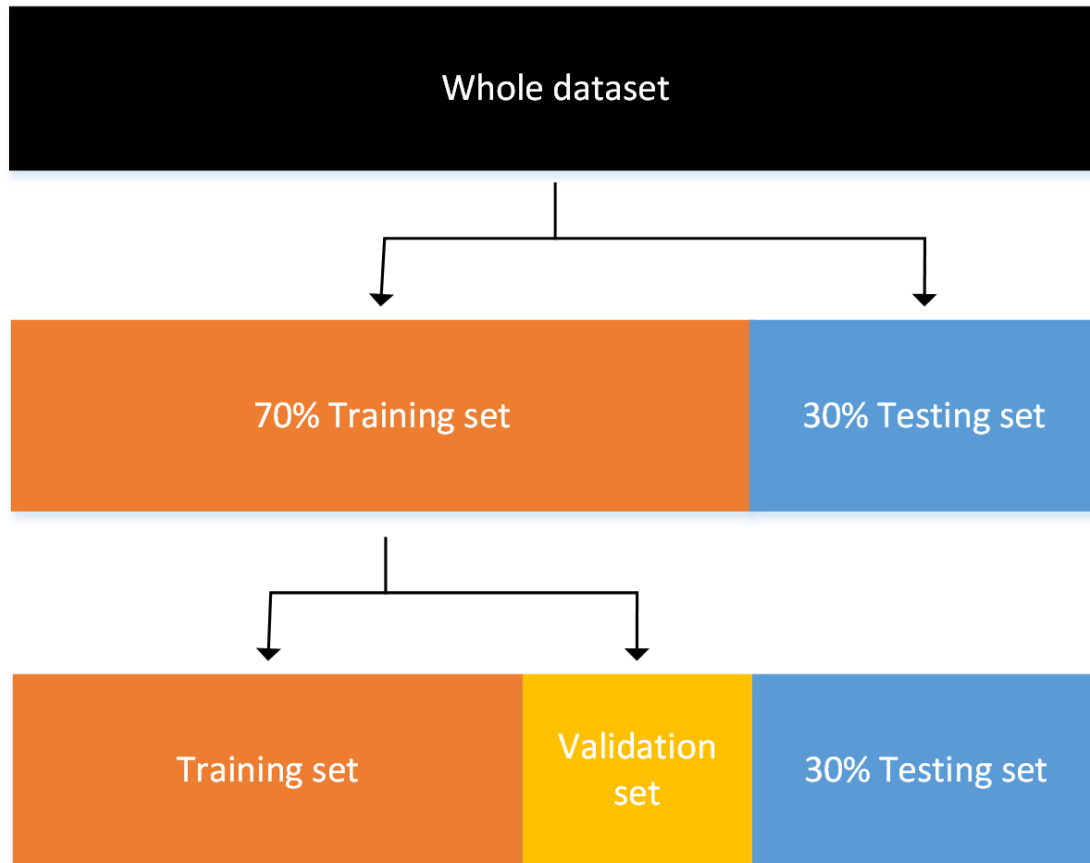
1. Get a reliable estimate of the classifier performance – i.e. how it will perform on new – so far unseen – data instances.
2. Compare your classifiers that you have developed – to decide which one is “the best”.

This helps you with model assessment and selection.



Remark: Whenever you extend your training data, you should get a better classifier! If not, something is wrong :((not representative data, wrong model, setting of learning parameters).

Main strategy to test performance



Classifier performance evaluation

Classifiers are learned (**trained**) on a finite training dataset. Part of this training set is used for evaluation during training.

Trained classifier has to be **tested on a different test set** in order to evaluate the performance.

Usual way how to train and test the classifier is to divide the dataset into training and testing part. But there is some trade-off:

1. More training data gives better generalization.
2. More test data gives better estimate for the classification error.



Never evaluate performance on training data!

The conclusion would be optimistically biased - overoptimism.

Classifier performance evaluation

Partitioning of available finite set of data to training / test sets. Strategy of partitioning:

1. Hold out
2. Cross validation - typically used

Based on this partitioning, you can then compute the mean value of performance estimate, and standard deviation.

Remark:

Although the cross-validation is usually connected with classifiers, it is also used for evaluation of regression model.

Hold out method

Given data is **randomly** partitioned into two independent sets.

1. Training set (e.g., $2/3$ of data) for learning the classifier.
2. Test set (e.g., $1/3$ of data) is hold out for the accuracy estimation of the classifier.

Then: Repeat this random **hold out** k times; the accuracy is estimated as the average of the accuracies obtained during these two steps.

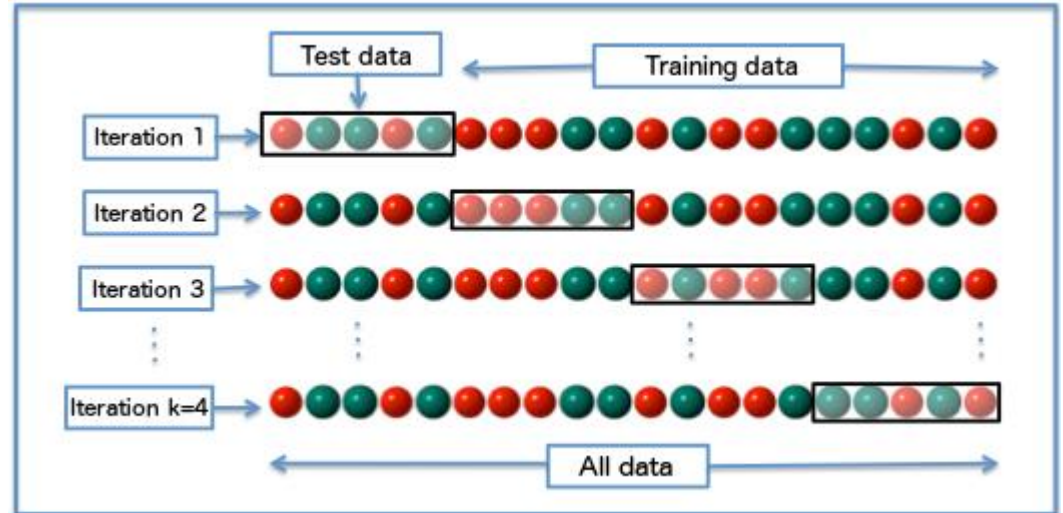
Overoptimism

Overoptimism occurs when exact or similar replicas of a given pattern exist in both the training and test sets. In this case, the classification performance in the test sets will be similar to the one obtained in the training sets, not because the model is able to correctly generalize to the test data, but rather because there are similar patterns in both training and test partitions.

In this context, overoptimism is associated to incorrect implementations of cross-validation approaches, when oversampling is used (see later).

K-fold cross validation

1. The training set is randomly divided into K disjoint sets of equal size where each part has roughly the same class distribution (the same number of samples in each class).
2. The classifier is trained K times, each time with a different set held out as a test set. Therefore, we have K estimates of train error.
3. The estimated error is the mean of these K errors. The standard deviation of error is also important.

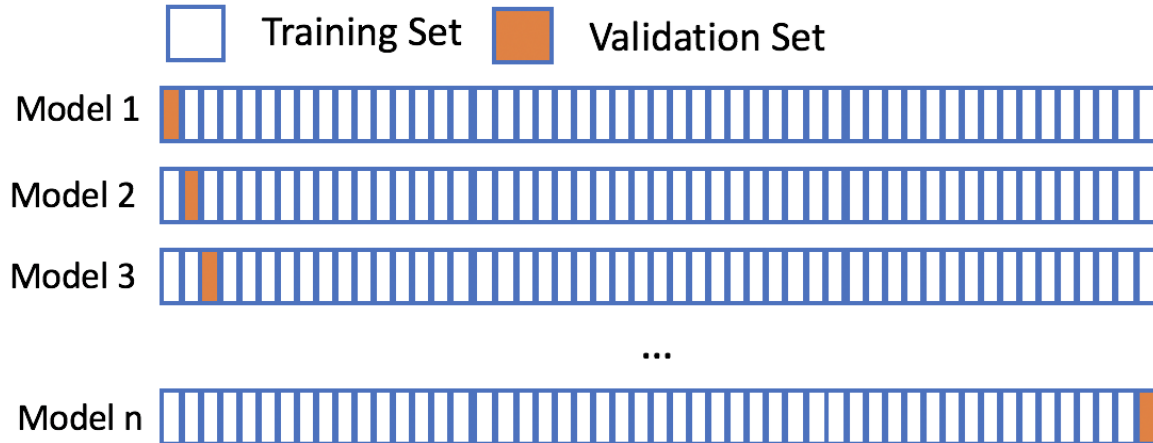


Leave-one-out cross validation

1. A special case of K-fold cross validation with $K = n$, where n is the total number of samples in the training multiset.
2. Totally n experiments are performed using $n - 1$ samples for training and the remaining sample for testing.

It is rather computationally expensive. Imagine that you have e.g. 10.000 samples.

Advantage: Uses maximum training set.



Unbalanced dataset

Why to deal with unbalanced dataset?

Unbalanced problems

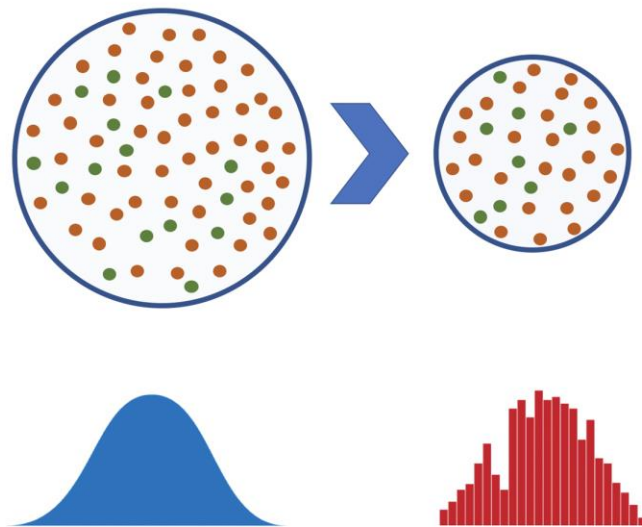
Classes have often unequal frequency and it is difficult to have the same number of data/samples/examples in both (all) classes.

- Medical diagnosis: 95 % healthy, 5% disease.
- Security: 99.999 % of citizens are not terrorists.

The problem arises:

How should we train classifiers and evaluated them for unbalanced problems?

Remark about
data sampling



A **population** is the collection of all subjects of interest.

A **sample** is a subset of the population of interest.

How to cope with unbalanced dataset

- A. Try to sample/collect more data - simple but not realistic in all cases.
- B. Test different classification algorithms – some methods can handle easier this problem, e.g. random forest-based methods.
- C. Change (use more) performance metric to get better view of the problem.
- D. Data resampling.
- E. Generate synthetic samples.
- F. Penalization.

Unbalanced dataset - change performance metric

Precision or Accuracy (see later) is not always good measure of performance. There are different measures to use. Sometimes very case-specific (e.g. Object detection in images).

Using various measures (more than one) can provide more insight into the performance of the model than traditional classification accuracy.

Unbalanced dataset - data resampling

Usually two main approaches are used:

1. **undersampling** - removing majority examples
2. **oversampling** - replicates the minority examples

Researchers/Data analysts often use oversampling procedures since they are capable of balancing class distributions without ruling out potentially critical majority examples.

Example - undersampling

Consider two-class problem. Our dataset contains one class (A,B,C,D,E) and second class (1,2,3,4,5,6,7,8,9,10). Therefore the unbalance is 50:100.



Undersampling means that we select and throw away set of samples from the second class to make the classes balanced, for example (1, 5, 7, 8, 10). This can be dangerous because we can throw away “important” samples.

(Wrong) Example - oversampling 😞

Consider the same 2-class example. Now, perform oversampling first and then do the cross-validation (CV).

During oversampling, one or several copies of some sample can appear in the new (oversampled) dataset (depending on the level of unbalance and strategy of sample selection). Consequently, it is possible that copies of the same patterns appear in **both the training and test sets**, making this design subjected to **overoptimism**.

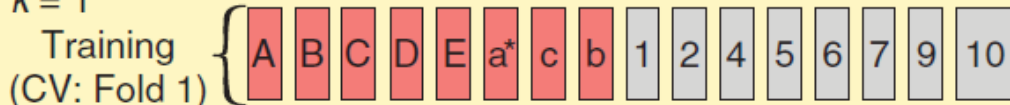
CV After Oversampling



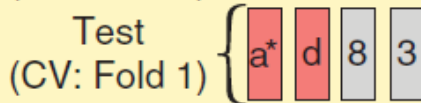
for Fold = 1 to all the k Partitions of CV

$k = 1$

Training
(CV: Fold 1)

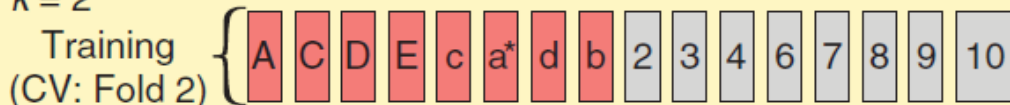


Test
(CV: Fold 1)

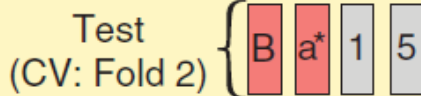


$k = 2$

Training
(CV: Fold 2)



Test
(CV: Fold 2)



(Correct) Example - oversampling 😊

In the second approach, the oversampling procedure is performed during cross-validation: the dataset is first divided into k stratified partitions and only the training set (corresponding to $k - 1$ partitions) is oversampled. In this scenario, the patterns included in the test set are never oversampled or seen by the model in the training stage, thus allowing a proper evaluation of the model's capability to generalize from the training data.

CV During Oversampling

for Fold = 1 to all the k Partitions of CV

$k = 1$

Training (CV: Fold 1) { B C D E 3 4 5 6 7 8 9 10

Test (CV: Fold 1) { A 1 2

[Oversampling]

Training (CV: Fold 1) { B C D E c b c e 3 4 5 6 7 8 9 10

Test (CV: Fold 1) { A 1 2

$k = 2$

Training (CV: Fold 2) { A C D E 1 2 5 6 7 8 9 10

Test (CV: Fold 2) { B 3 4

[Oversampling]

Training (CV: Fold 2) { A C D E a d c d 1 2 5 6 7 8 9 10

Test (CV: Fold 2) { B 3 4

<https://ieeexplore.ieee.org/document/8492368> (...)

Oversampling - note

This copying is called as **ROS** (Random Oversampling) is the simplest of oversampling techniques, where the existing minority examples are replicated until the class distribution is balanced. This approach is often criticised since it does not introduce any new information to the data (the oversampled examples are mere copies of the original data points) and may lead to overfitting (even if cross-validation is performed properly).

Unbalanced dataset - Generate synthetic samples

There are different approaches how to generate new/artificial samples (more details in this review Santos et al. <https://ieeexplore.ieee.org/document/8492368>)

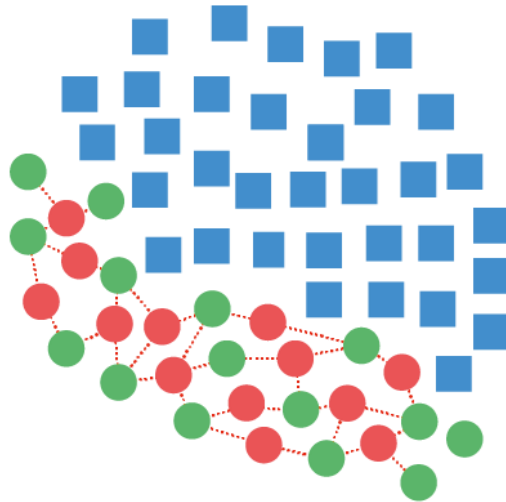
SMOTE - Synthetic Majority Oversampling Technique - works by generating synthetic minority examples along the line segments joining randomly chosen G minority examples and their k -nearest minority class neighbors. G is the number of minority examples to oversample in order to obtain the desired balancing ratio between the classes, and along with the value of k , it can be specified by the user.

SMOTE

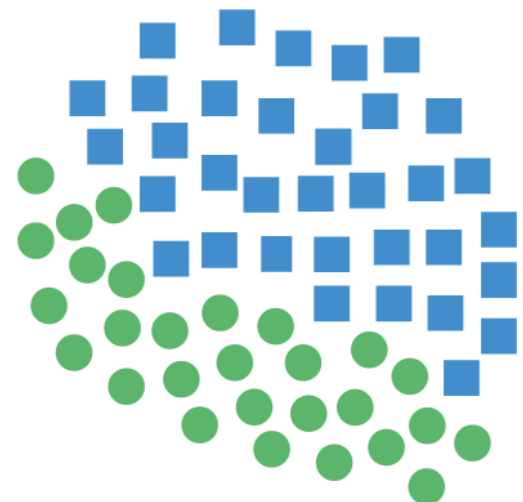
Synthetic Minority Oversampling Technique



Original Dataset

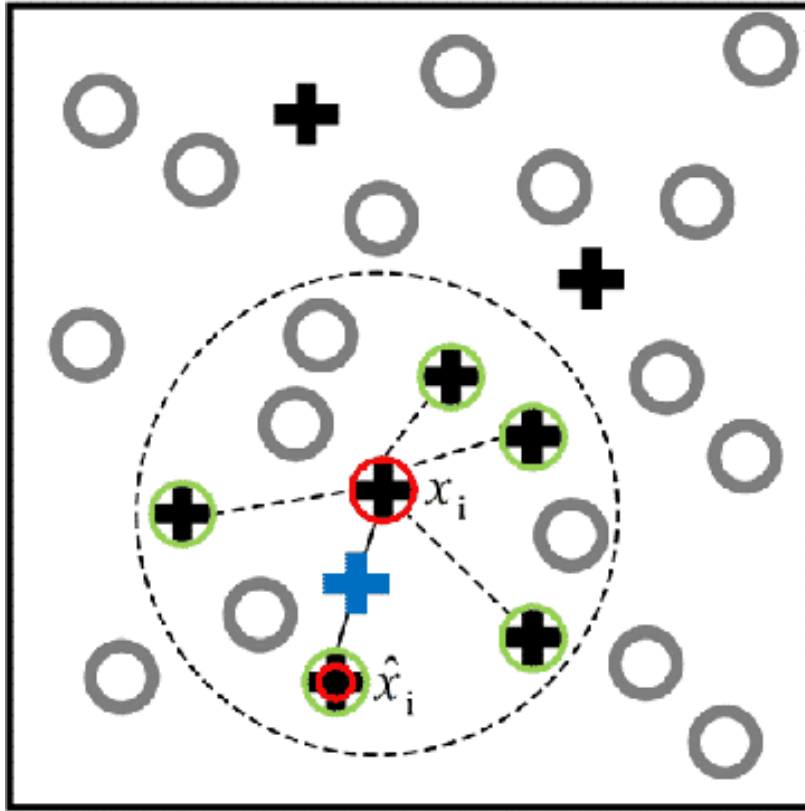


Generating Samples



Resampled Dataset

SMOTE



Majority class samples



Minority class samples



Randomly selected sample



K-nearest neighbours, here $K=5$



Randomly selected sample from the 5 neighbours



Generated synthetic minority sample

Oversampling - SMOTE

SMOTE will then **generate a new synthetic sample s** according to $s = x + r(x-v)$, where x is the minority sample to oversample, v is one of its chosen nearest neighbors and r is called a gap, in this case, a random number between 0 and 1. By generating similar examples to the existing minority points, SMOTE creates larger and less specific decision boundaries that increase the generalization capabilities of classifiers, therefore increasing their performance.

ADASYN - Adaptive Synthetic Sampling Approach

This procedure is the same as SMOTE; the only difference is that 'harder' minority examples are replicated more often. 'Harder' means more difficult to learn, i.e. samples lying close to majority samples.

ADASYN determines a weight w_i for each minority example, defined as the normalized ratio of majority examples N_i among its k nearest neighbors:

ADASYN

- Calculate the ratio of minority to majority examples using: $d = \frac{N_{minority}}{N_{majority}}$
- Calculate the total number of synthetic minority data to generate:

$$G = \beta (N_{majority} - N_{minority})$$

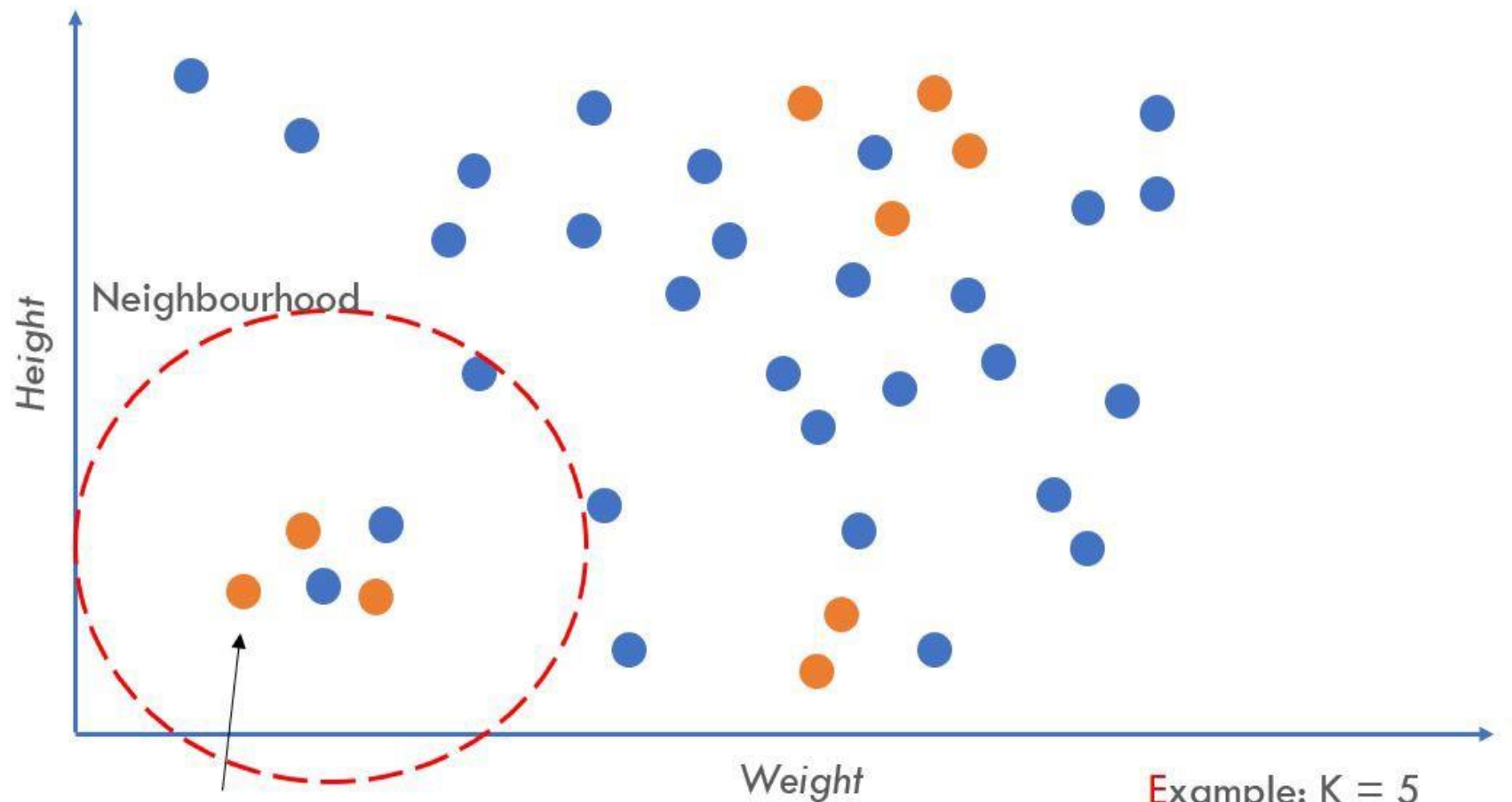
where β defines balance between number of samples in each class.

- Find the k-NN samples of each minority example and calculate the r_i value:

$$r_i = \frac{\text{Number of majority samples}}{k}$$

- Normalize the r_i values so that the sum of all r_i^{norm} values equals to 1.
- Calculate the amount of synthetic examples to generate for each minority sample (in its neighbourhood):

$$G_i = G \cdot r_i^{norm}$$



Point of interest

$K = 5$

Example: $K = 5$

$r1 = 3 / 5$

$r1 = 0.6$

Unbalanced dataset - penalization

- Penalization imposes an additional cost on the model for making classification mistakes on the minority class during training. The result is that these penalties can force (*bias*) the model to pay more attention to the minority class.
- There are penalized versions of algorithms such as penalized-SVM and penalized-LDA.

Classifier performance assessment

Criterion function to assess classifier performance

Typically, two quantities are evaluated:

- **Accuracy** is the percent of correct classifications.
- **Error rate** = is the percent of incorrect classifications.

It's obvious that:

$$\text{Accuracy} = 1 - \text{Error rate}$$

Other characteristics derived from the **confusion matrix**.

Predicted Class

Confusion matrix

		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$

Confusion matrix - example

		Predicted Class	
		Spam	Non-Spam
Actual Class	Spam	TP=45	FN=20
	Non-Spam	FP=5	TN=30

Confusion matrix - multiclass, example

▼ TRAINING METRICS - CONFUSION MATRIX

	0	1	2	3	4	5	6	7	8	9	Error	Rate
0	902	0	10	5	1	12	3	3	7	3	0.0465	44 / 946
1	0	1057	8	4	2	6	4	6	14	7	0.0460	51 / 1,108
2	14	11	826	25	23	5	17	17	25	4	0.1458	141 / 967
3	7	6	15	900	2	39	2	16	33	11	0.1271	131 / 1,031
4	1	3	13	1	893	3	5	7	3	52	0.0897	88 / 981
5	14	7	7	25	13	814	29	3	26	15	0.1459	139 / 953
6	8	5	25	1	21	18	875	3	7	4	0.0951	92 / 967
7	6	10	10	9	9	1	0	893	0	44	0.0906	89 / 982
8	9	21	8	24	13	42	10	5	822	31	0.1655	163 / 985
9	7	6	4	6	40	5	0	39	11	885	0.1176	118 / 1,003
Total	968	1126	926	1000	1017	945	945	992	948	1056	0.1064	1,056 / 9,923

Accuracy and Error rate

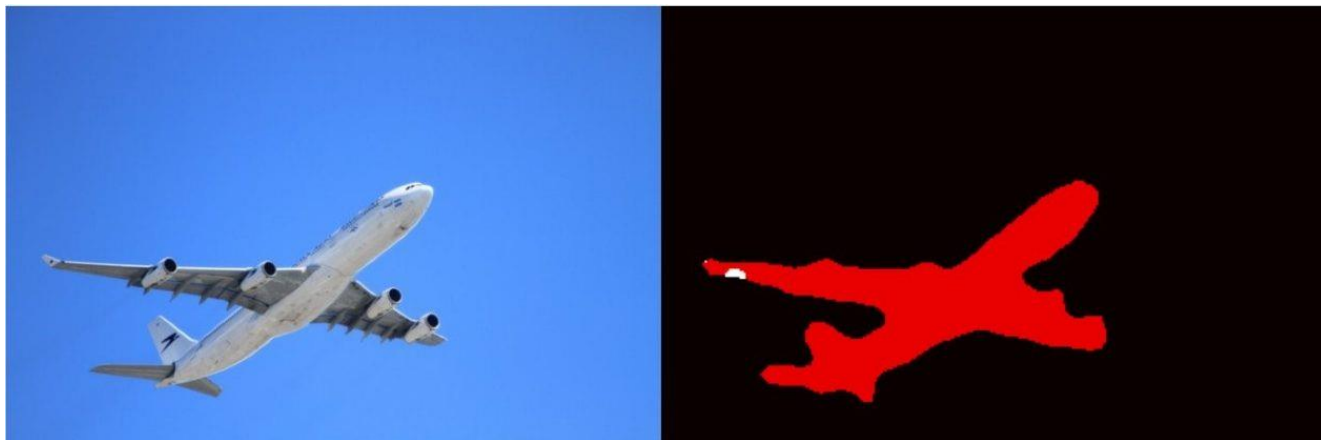
1. Classification Accuracy, **Acc** - percent of correct classification.
2. Error rate, **Err** - percent of incorrect classification.

$$Acc = (100) \cdot \frac{TP+TN}{TP+TN+FP+FN}$$

$$Err = 1 - Acc$$

Precision, Recall, F1-measure

These three measures don't care about number of true negatives (TN). This is important in some applications, where true negatives has no sense. Consider application - image segmentation of aircraft. There is around 15% of pixels inside the aircraft area (true positive) and the rest pixels (85%) belongs to non-aircraft area (true negative). Therefore, using measures with true negative, e.g. specificity ($TN/(TN+FP)$), would not be useful.



Precision

Positive prediction value (PPV) or precision:

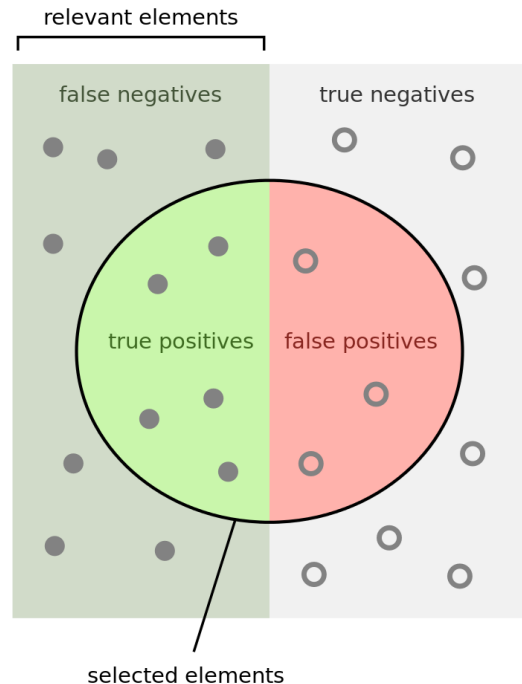
$$\textbf{Precision} = TP / (TP + FP) =$$

Number of positive correctly classified samples

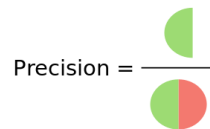
Total number of positive predicted samples

It tells us, what proportions of all the positive predictions are actually positive.

For example consider task of spam detection and the precision is 90%. This means that we correctly classified/detect 90% of relevant items - spam emails (10% of non-spam will be considered as a spam).

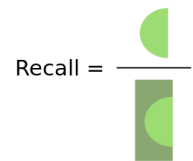


How many selected items are relevant?



Precision =

How many relevant items are selected?



Recall =

Recall

Recall known as Sensitivity, True positive rate (TPR) or hit rate:

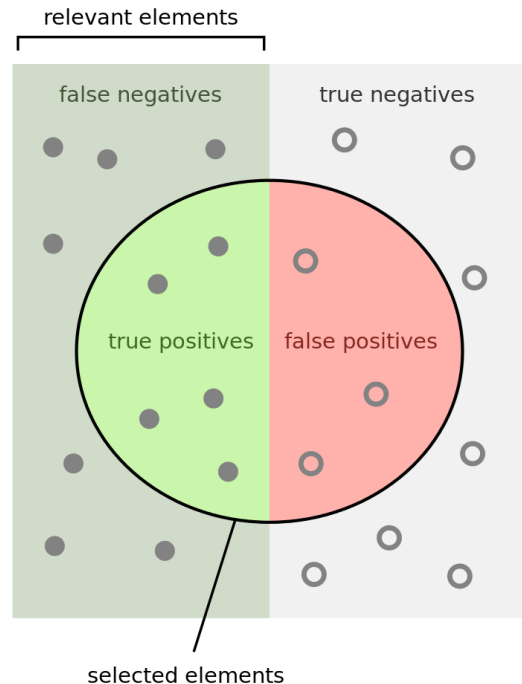
$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN}) =$$

Number of positive correctly classified samples

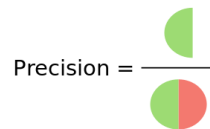
Total number of positive samples

It tells us, out of all the positive cases, how many positive cases does the model identify

For example consider task of spam detection and the recall is 90%. This means that we miss 10% of spam (10% of spam will be considered as a non-spam).

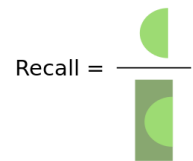


How many selected items are relevant?



Precision =

How many relevant items are selected?



Recall =

F_1 - measure

The F_1 -measure (or F-measure/score) is a *harmonic mean* of precision and recall, i.e.:

$$F_1 = \frac{1}{\frac{\frac{1}{Pr} + \frac{1}{Re}}{2}} = \frac{2}{\frac{1}{Pr} + \frac{1}{Re}} = \frac{2 \times Pr \times Re}{Pr + Re}$$

The harmonic mean is more intuitive than the arithmetic mean when computing a mean of ratios.

Suppose that you have a fingerprint recognition system and its precision and recall be 1.0 and 0.2, respectively. Intuitively, the total performance of the system should be very low because the system covers only 20% of the registered fingerprints, which means it is almost useless. The F_1 -score is 1/3, whereas the arithmetic mean of 1 and 0.2 is 0.6.

F_β measure

The full definition of the F-measure is given as:

$$F_\beta = \frac{(\beta^2 + 1) \times Pr \times Re}{\beta^2 Pr + Re}$$

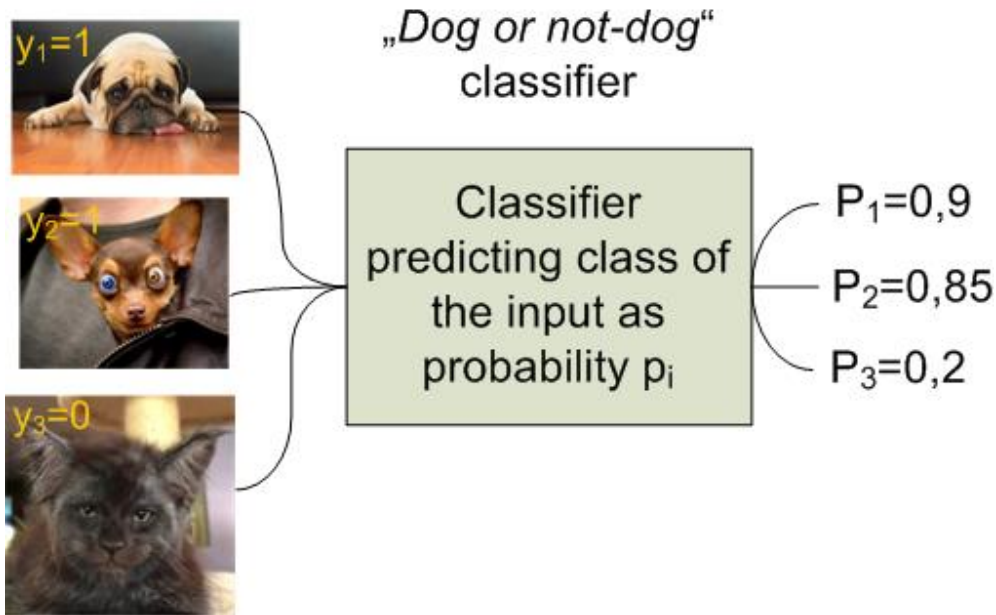
β is a parameter that controls a balance between Pr and Re . When $\beta = 1$, F_1 comes to be equivalent to the harmonic mean of Pr and Re . If $\beta > 1$, F becomes more *recall-oriented* and if $\beta < 1$, it becomes more *precision oriented*, e.g., $F_0 = Pr$.

The F_1 -score is also known as the Sørensen–Dice coefficient or Dice similarity coefficient or just Dice coefficient.

Logarithmic Loss, Log Loss

Log loss measures the uncertainty of the probabilities of your model by comparing them to the true labels. It is used for two or more classes.

Consider that our classifier predict the input in term of probabilities - we have labels (0 or 1), but classifier's output is a prediction. So how do we calculate an error of our model?



$$LogLoss = -\frac{1}{n} \sum_{i=1}^n (y_i \log p_i + (1 - y_i) \log(1 - p_i))$$

Log Loss

Consider four cases (y_i denotes class (0 or 1) and p_i denotes predicted probability):

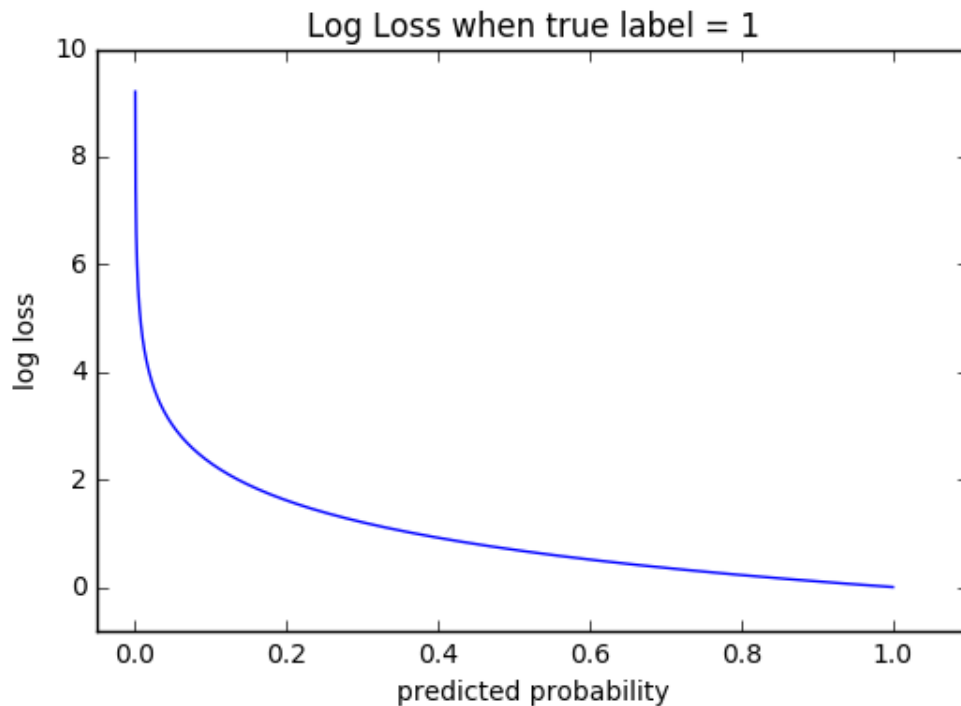
- 1) $y_i = 1$ and $p_i = \text{high}$ (close to 1), this means that we are **predicting well**. We evaluate this case as $y_i \cdot \log(p_i)$, which will be **small**.
- 2) $y_i = 1$ and $p_i = \text{low}$ (close to 0), this means that we are **predicting wrong**. We evaluate this case as $y_i \cdot \log(p_i)$, which will be **high**.
- 3) $y_i = 0$ and $p_i = \text{low}$ (close to 0), this means that we are **predicting well**. We evaluate this case as $(1 - y_i) \cdot \log(1 - p_i)$, which will be **small**.
- 4) $y_i = 0$ and $p_i = \text{high}$ (close to 1), this means that we are **predicting wrong**. We evaluate this case as $(1 - y_i) \cdot \log(1 - p_i)$, which will be **high**.

$$\text{LogLoss} = -\frac{1}{n} \sum_{i=1}^n (y_i \log p_i + (1 - y_i) \log(1 - p_i))$$

Log Loss

The graph shows the range of possible loss values given a true observation (isDog, $y_i=1$). As the predicted probability approaches 1, log loss slowly decreases. However, as the predicted probability decreases, the log loss increases rapidly.

LogLoss can be used also for multiclass classification (One v. Rest).



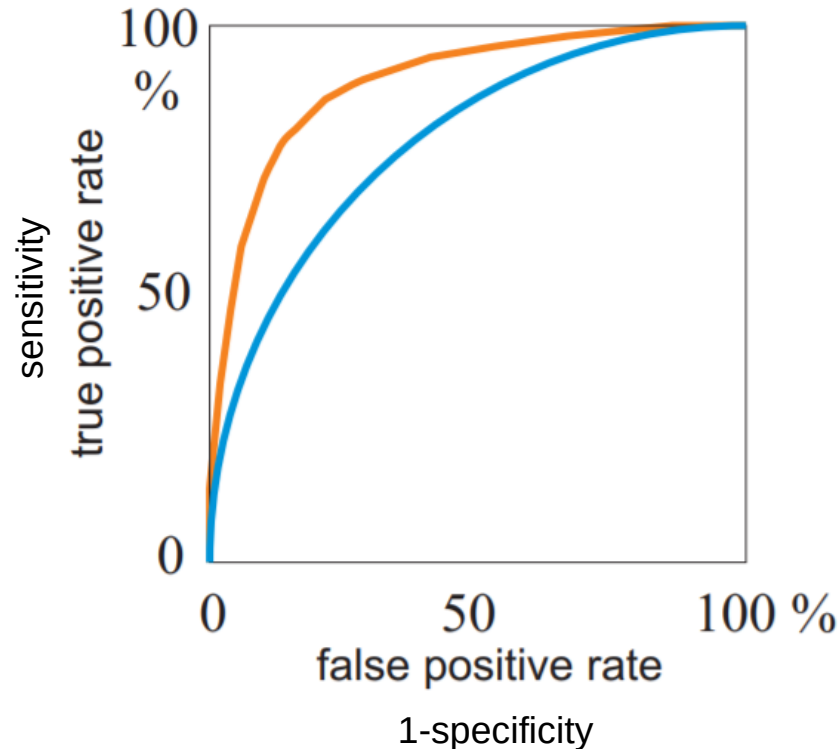
ROC analysis of classification

ROC analysis

ROC (Receiver Operating Curve) graphs are two-dimensional graphs in which **TP rate (sensitivity)** is plotted on the Y axis and **FP rate (1-specificity)** is plotted on the X axis. An ROC graph depicts relative tradeoffs between benefits (true positives) and costs (false positives).

Characterizes degree of overlap of classes.

Decision is based on a single threshold θ (called also operating point).



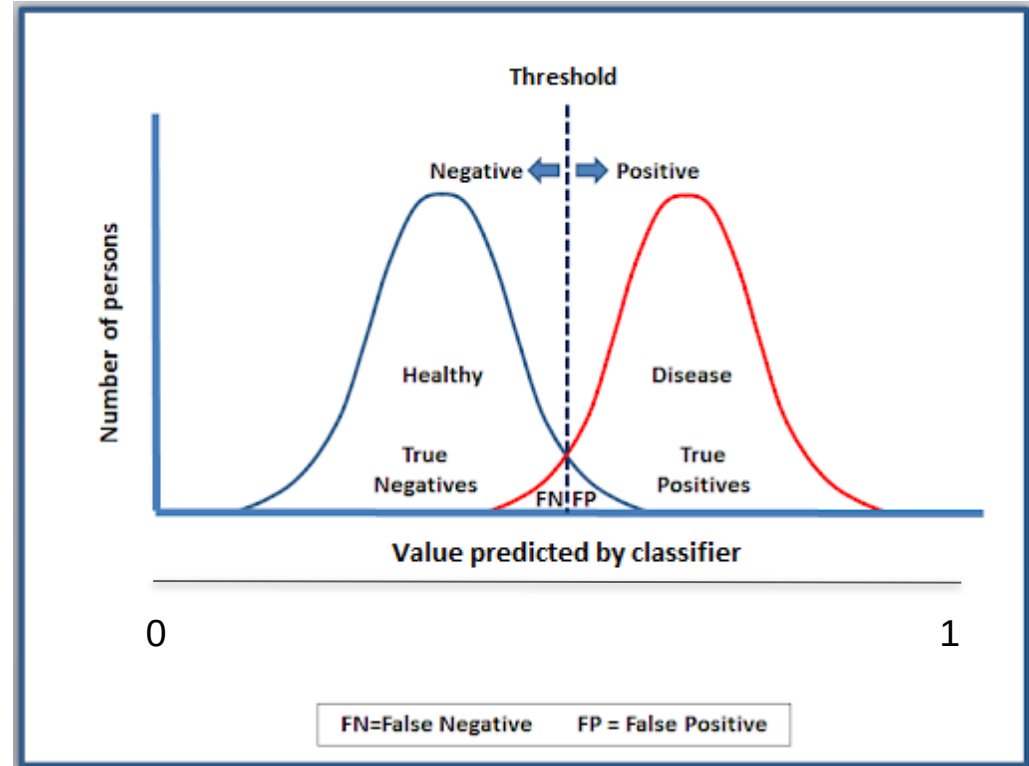
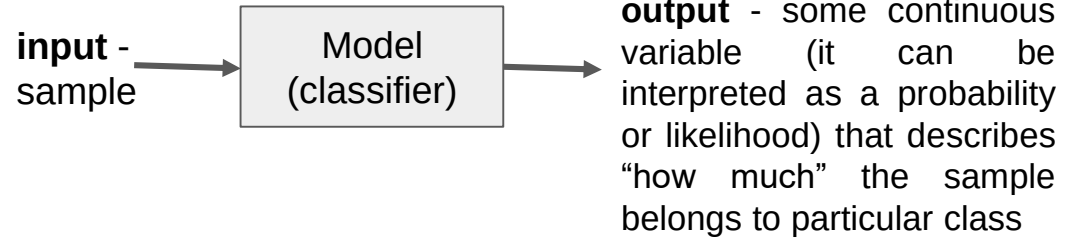
More information: Fawcet: Introduction to ROC analysis,

<https://www.sciencedirect.com/science/article/abs/pii/S016786550500303X>

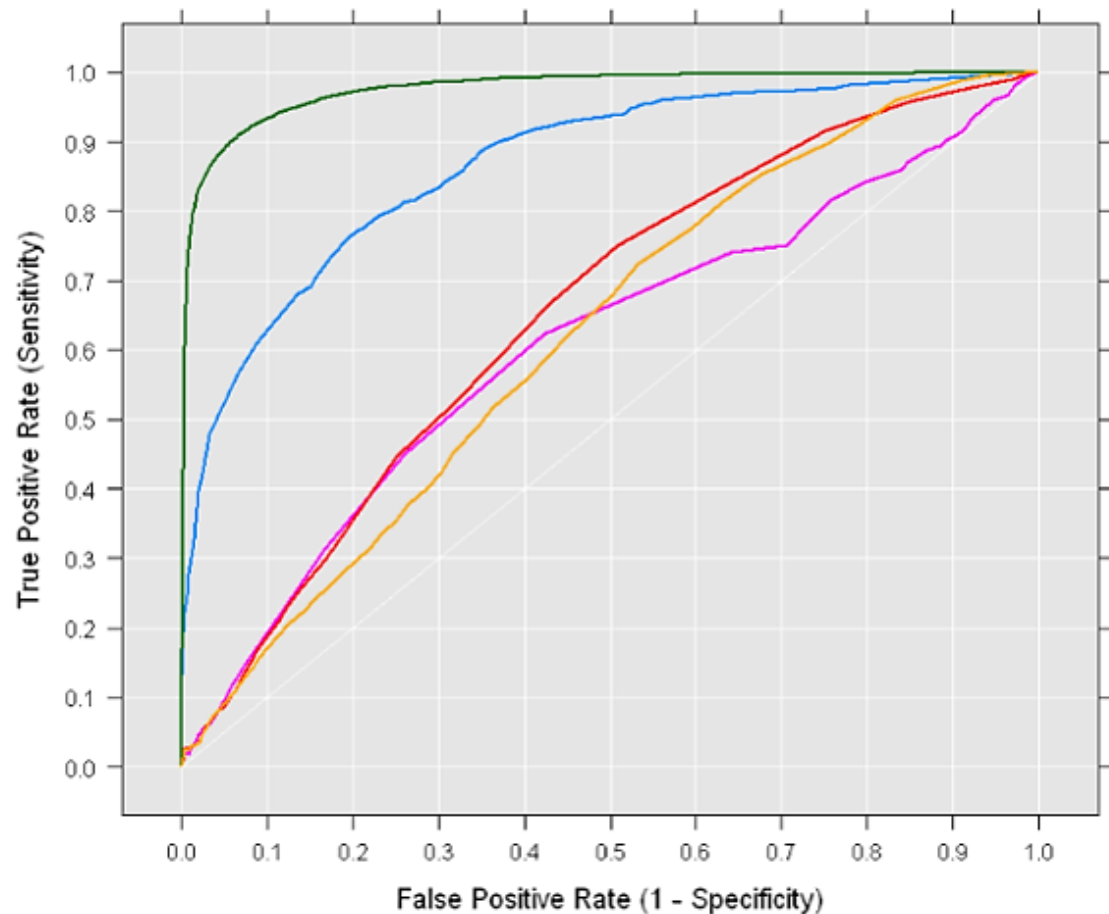
ROC curve

Consider probability distribution function for two classes (healthy/disease, cat/dog...). These are represented by red and blue Gaussian curve in the image on the right. The y-axis represents the output of the classifiers, which is continuous and must be thresholded to classify input data into particular class.

Moving the threshold will change the number of TP, TN, FN, FP and change the sensitivity and specificity value and therefore it will “generate” single point on the ROC curve.

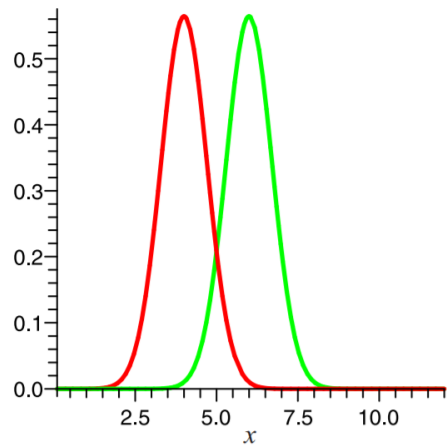


ROC curve

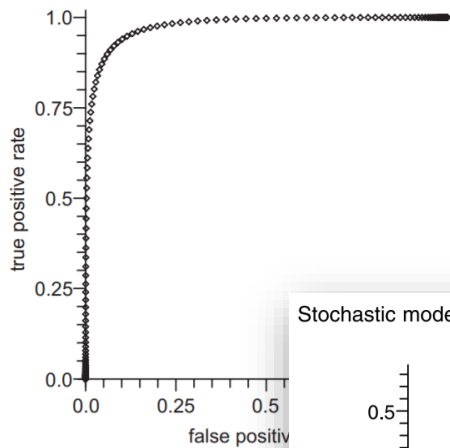


ROC - Gaussians, same variances, different overlap

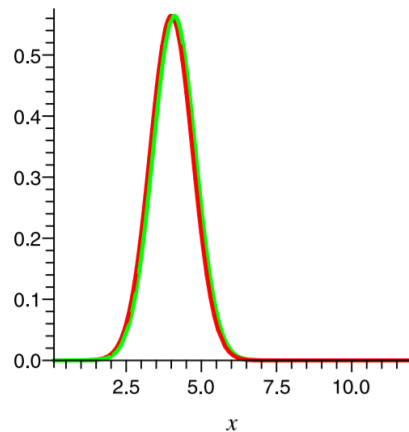
Stochastic model, Gaussians, equal variances



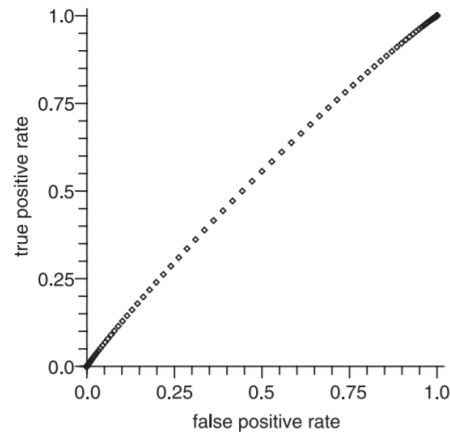
ROC curve



Stochastic model, Gaussians, equal variances

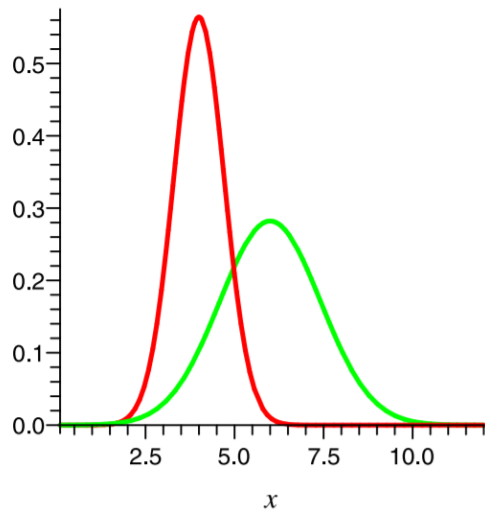


ROC curve

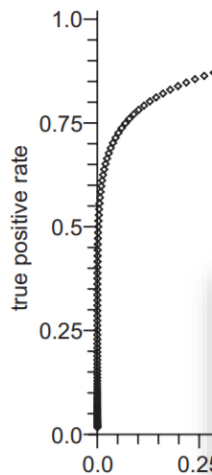


ROC - Gaussians, different variances and overlap

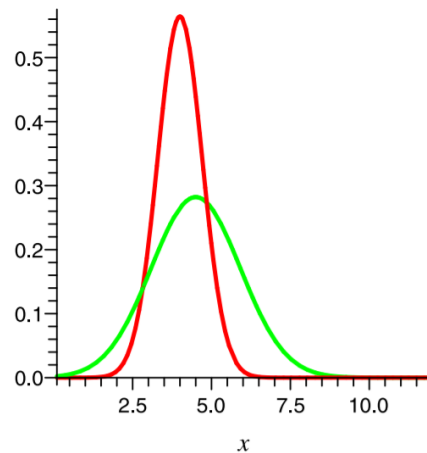
Stochastic model, Gaussians, different variances



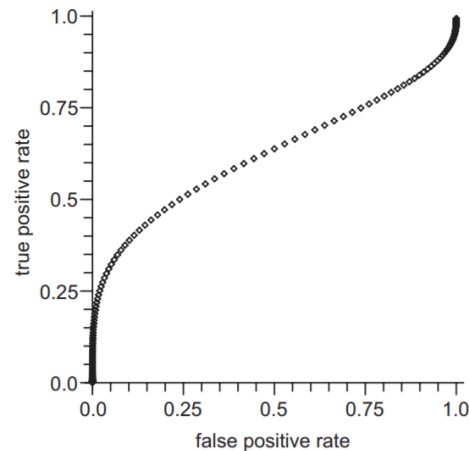
ROC curve



Stochastic model, Gaussians, different variances



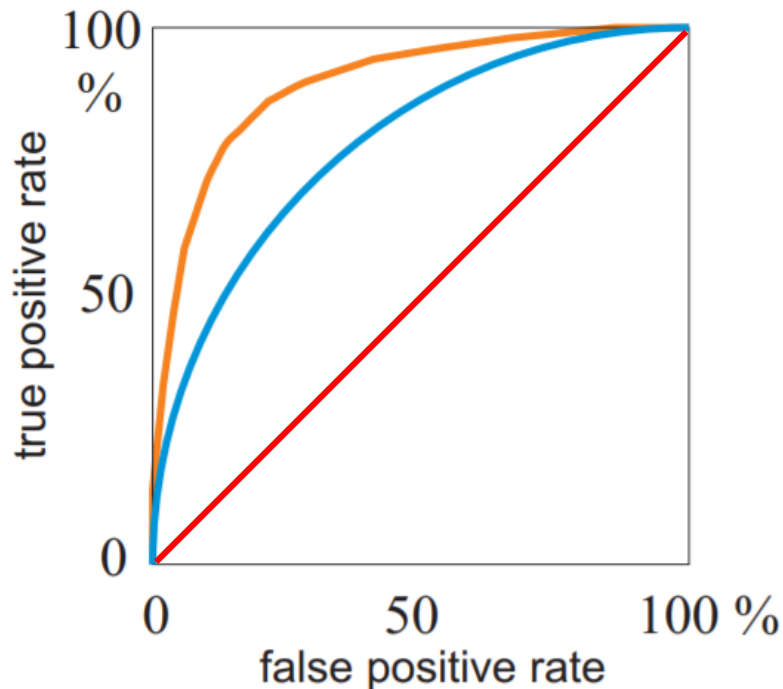
ROC curve



ROC - AUC

A classifier with the Red line is guessing the class randomly. Closer the ROC curve gets to top-left part of the chart, better the classifier is (yellow is better than blue).

Area under the curves (**AUC**) is the area below these ROC curves. Therefore, in other words, AUC is a great indicator of how well a classifier functions.

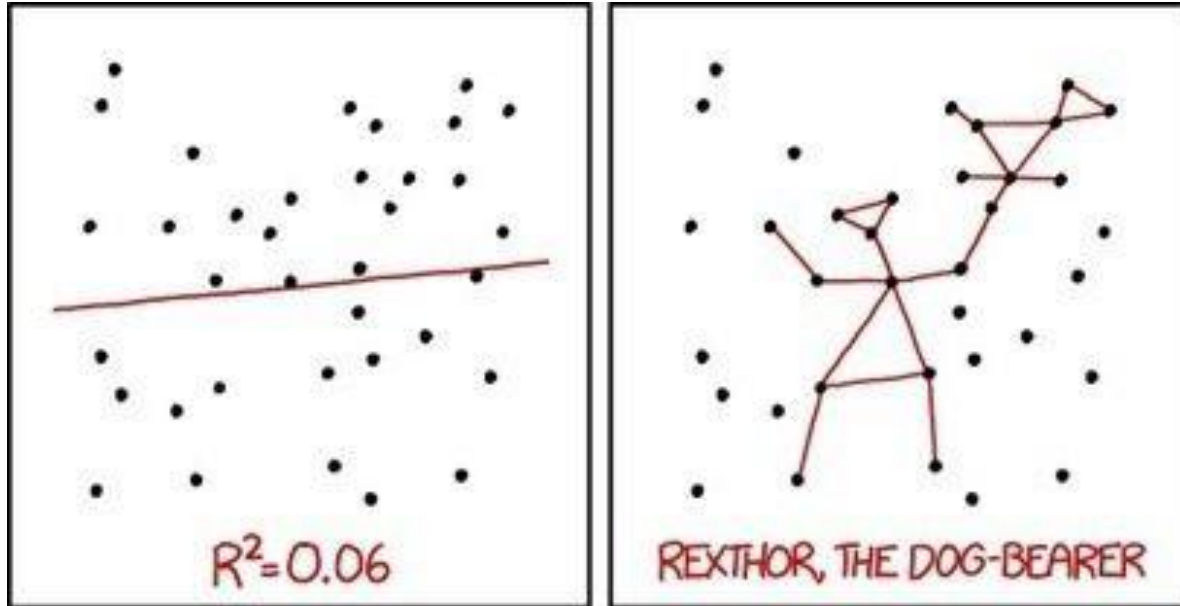


Conclusion - how to evaluate classifier?

1. **Classification Accuracy** - percent of **correct** classification.
2. **Error rate** - percent of **incorrect** classification.
3. **Precision and recall (sensitivity).**
4. **F1-score**
5. **LogLoss**
6. **Area Under ROC Curve**
7. **Confusion Matrix**

		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$

Regression performance

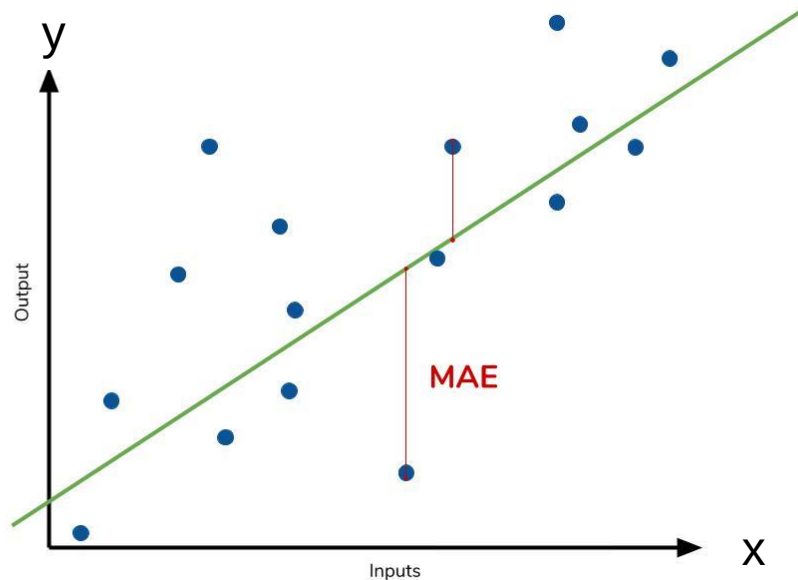


I DON'T TRUST LINEAR REGRESSIONS WHEN IT'S HARDER
TO GUESS THE DIRECTION OF THE CORRELATION FROM THE
SCATTER PLOT THAN TO FIND NEW CONSTELLATIONS ON IT.

Performance of regression

The **Mean Absolute Error (MAE)** - calculate the residual for every data point, taking only the absolute value:

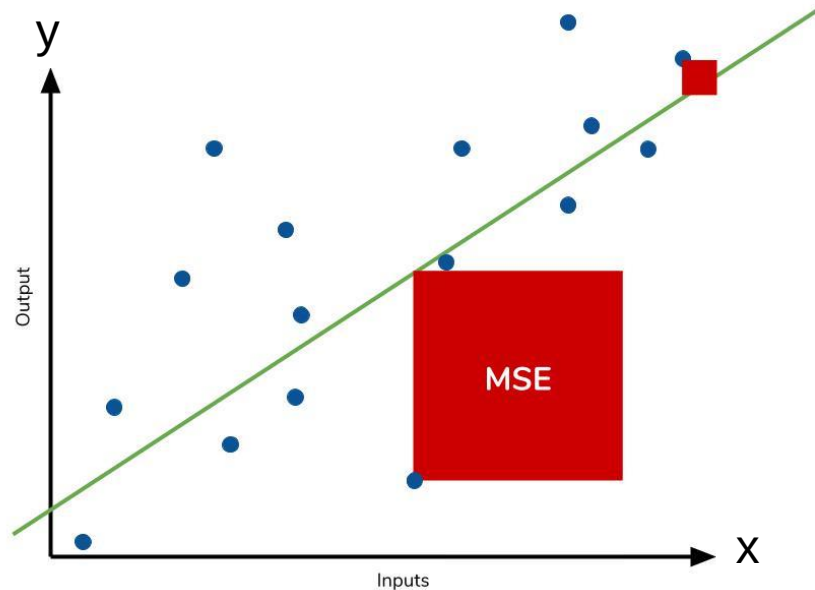
$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \tilde{y}_i|$$



Performance of regression

The **Mean Square Error (MSE)** is similar to MAE, but squares the difference before summing them all instead of using the absolute value.

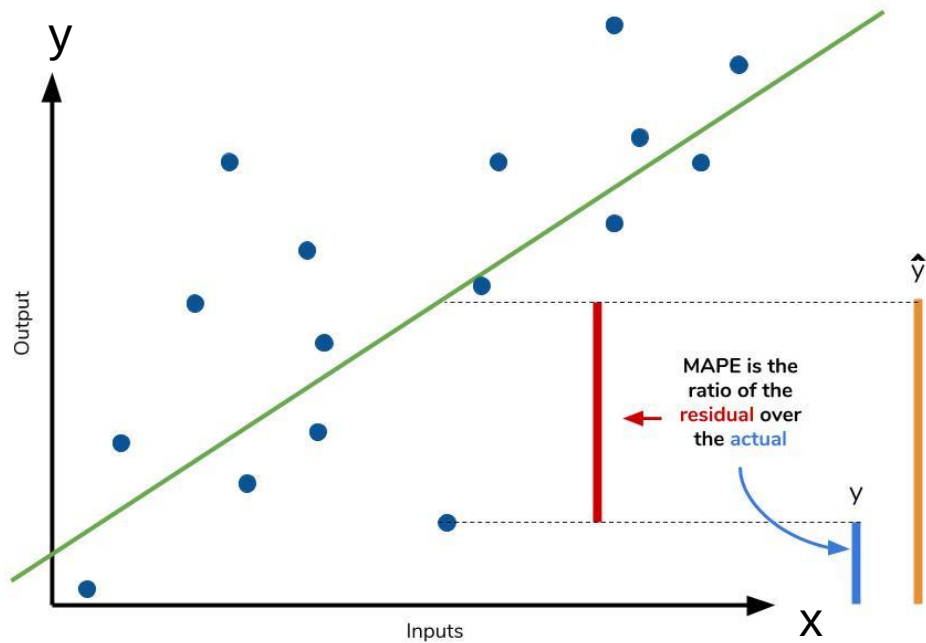
$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \tilde{y}_i)^2$$



Performance of regression

The **Mean Absolute Percentage Error** (MAPE) is the percentage equivalent of MAE, except that the error is relative to give output value y :

$$MAPE = \frac{1}{N} \sum_{i=1}^N \left| \frac{y_i - \tilde{y}_i}{y_i} \right|$$



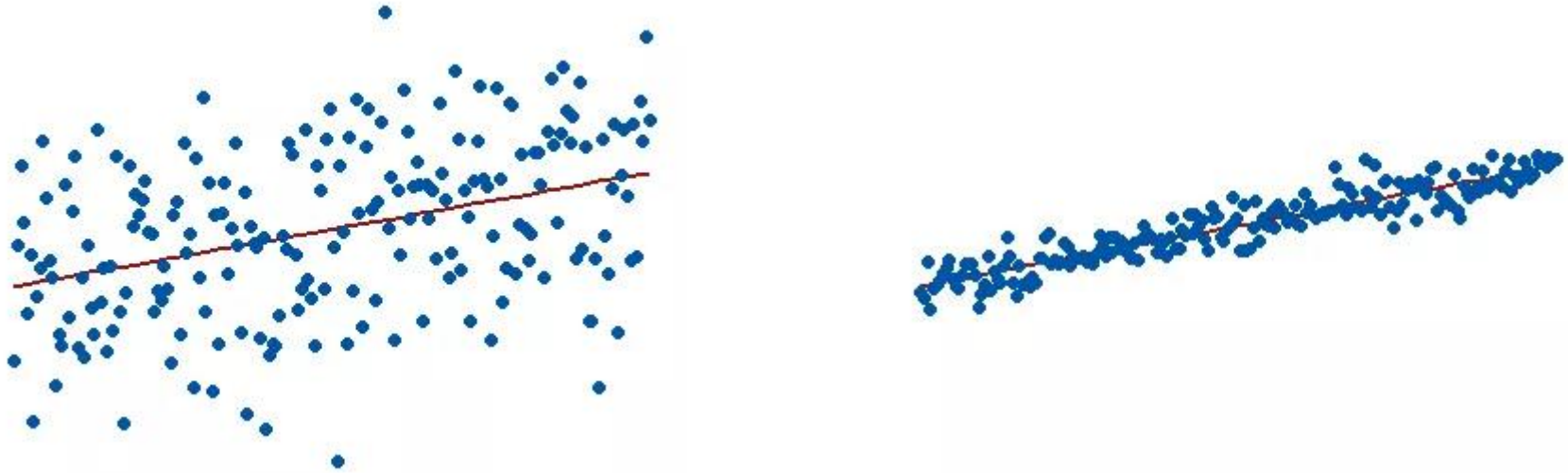
Performance of regression

Coefficient of determination, Goodness of fit, R^2 - quantifies the quality of regression and can be also interpreted as variation of the model with respect to total variation of the data.

We can compute the sum of squared error of regression model (SSE) as a sum of squared differences between the data (y) and prediction ($\hat{y}$). This will gives us some information about the quality of regression. But it will be depending on the data values. Therefore we will *normalize* it by variation (i.e. variance) of the data. The final R^2 value is then defined as:

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \tilde{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y}_i)^2}$$

R^2 value - example



R^2 for the regression model on the left is 15%, and for the model on the right it is 85%. When a regression model *covers* more variance, the data points are closer to the regression line/curve. In practice, a regression model with an R^2 of 100% is impossible. In that case, the fitted values equal the data values and, consequently, all of the observations fall exactly on the regression line.

R^2 value

R^2 value can be used for comparison of different models.

The values above 0.7 is usually desirable, between 0.8 - 0.9 is considered as good and above 0.9 as an excellent.

How to evaluate regressor?

1. Mean Absolute Error - MAE
2. Mean Squared Error - MSE
3. Mean Absolute Percentage Error - MAPE
4. R^2 value